YFB Derivation Notes

OpenAI Codex

2026-05-08

1 Big Picture

The cleanest way to explain YFB is:

- **LB model:** survival is driven by the latent patient scores L
- **YFB model:** survival is driven by the observed projection scores YF

So the main derivation move is not to invent a wholly different matrix factorization. It is:

1. keep the same genomics factorization $Y \approx LF^\top$
2. keep the same Cox survival layer
3. replace the survival predictor
   - from $\eta_i = l_i^\top \beta$
   - to $\eta_i = (y_i^\top F)\tilde{\beta}$

This is exactly how the implementation frames it in fit_cox_on_yf.R.

2 Why YFB Was Derived From LB

The motivating problem is the **train/test mismatch** in LB.

In LB:

- during training, survival uses the fitted latent scores $E[L]$
- during testing, there is no latent L_{test}
- test data must instead be projected onto F to get a surrogate score

That means:

- training predictor: $\eta_{\text{train}} = L\beta$
- test predictor: $\eta_{\text{test}} \approx Y_{\text{test}}F(\cdots)$

Those are not literally the same object.

The YFB derivation removes that mismatch by forcing the model to use the same predictor form at train and test time:

- training: $\eta = YF\tilde{\beta}$
- testing: $\eta_{\text{test}} = Y_{\text{test}}F\tilde{\beta}$

3 Conceptual Derivation From LB to YFB

Start from the usual LB model:

$$Y = LF^\top + E, \qquad E_{ij} \sim N(0, \tau_j^{-1})$$

with Cox survival:

$$h(t_i \mid l_i) = h_0(t_i) \exp(l_i^\top \beta)$$

So the LB linear predictor is:

$$\eta_i^{LB} = l_i^\top \beta$$

Now ask how latent scores connect to observed data. If F is fixed, a natural score-like projection of sample i onto factor k is:

$$z_{ik} = y_i^\top f_k$$

or in matrix form:

$$Z_F = YF$$

This is an observed score matrix built directly from the data and factor matrix.

YFB then says: instead of making hazard depend on the latent L, make it depend on these observed scores:

$$\eta_i^{YFB} = z_i^\top \tilde{\beta} = (y_i^\top F)\tilde{\beta}$$

Equivalently,

$$\eta^{YFB} = YF\tilde{\beta}$$

So the model becomes:

- genomics model unchanged: $Y \approx LF^\top$
- survival model changed: $\eta = YF\tilde{\beta}$

4 What This Changes Mathematically

This one substitution changes the dependency structure.

4.1 LB

- L appears in:
  - genomics likelihood
  - survival likelihood
- F appears in:
  - genomics likelihood only
- β appears in:
  - survival likelihood only

4.2 YFB

- L appears in:
  - genomics likelihood only
- F appears in:
  - genomics likelihood
  - survival likelihood, because $\eta = YF\tilde{\beta}$
- $\tilde{\beta}$ appears in:
  - survival likelihood only

That dependency switch is the core derivation result.

5 Clean Probabilistic Statement of YFB

A useful way to write the intended YFB model is:

$$Y_{ij} \mid L, F, \tau \sim N\left(\sum_{k=1}^{K} l_{ik} f_{jk}, \tau_j^{-1}\right)$$

and

$$h(t_i \mid y_i, F, \tilde{\beta}) = h_0(t_i) \exp\left(\sum_{k=1}^{K} (y_i^\top f_k) \tilde{\beta}_k\right)$$

Define:

$$Z_F = YF, \qquad z_{ik} = (YF)_{ik}$$

then survival is just:

$$h(t_i \mid Z_F) = h_0(t_i) \exp(z_i^\top \tilde{\beta})$$

That makes the survival covariates observed once F is known.

6 Why This Matters for Variational Inference

Under mean-field, the approximation remains:

$$q(L, F, \beta) = q(L)q(F)q(\beta)$$

But because the survival term changed:

- $q(L)$ becomes simpler
- $q(\beta)$ becomes easier to activate
- $q(F)$ becomes harder in principle, because F now participates in both model parts

7 Actual Update Equations

The YFB code still uses the same general strategy as LB:

- second-order Taylor expansion of the Cox partial likelihood
- convert each coordinate update into an EBNM problem
- update one factor at a time

Let:

- w_i be the diagonal Cox Hessian weights
- z_i be the Cox working response
- $\eta = Z_F \tilde{\beta}$

Then each coordinate update is an EBNM problem with pseudo-observation $x = B/A$ and pseudo-noise $s = 1/\sqrt{A}$.

7.1 $q(\beta_k)$ in LB vs YFB

This is the cleanest difference.

7.1.1 LB

For LB, the survival covariate is latent l_{ik}, so:

$$A^{LB}_{\beta,k} = \sum_i w_i E[l^2_{ik}]$$

$$B^{LB}_{\beta,k} = \sum_i w_i z^{-k}_i E[l_{ik}]$$

and then

$$x_k = \frac{B_{\beta,k}}{A_{\beta,k}}, \qquad s_k = A^{-1/2}_{\beta,k}$$

7.1.2 YFB

For YFB, the survival covariate is the observed projection score

$$Z_{F,ik} = (YF)_{ik}$$

so the same update machinery is reused with

- $EL_k \leftarrow ZF[,k]$
- $EL2_k \leftarrow ZF[,k]^2$

since $ZF[,k]$ is observed and has no posterior variance.

Therefore:

$$A^{YFB}_{\beta,k} = \sum_i w_i Z^2_{F,ik}$$

$$B^{YFB}_{\beta,k} = \sum_i w_i z_i^{-k} Z_{F,ik}$$

This is why YFB can avoid the earlier "beta stuck at zero" failure mode in principle: the survival covariates are already present from the current F, rather than waiting for uncertain L to align first.

7.2 $q(L_k)$ in YFB

Because L no longer appears in survival, the YFB $q(L_k)$ update is pure genomics.

$$A_{L,k} = \sum_j \tau_j E[f^2_{jk}]$$

$$B_{L,k}(i) = \sum_j \tau_j R^{-k}_{ij} E[f_{jk}]$$

Then for each patient i:

$$x_{L,ik} = \frac{B_{L,k}(i)}{A_{L,k}}, \qquad s_{L,k} = A^{-1/2}_{L,k}$$

and an EBNM problem is solved across the entries of column L_k.

So the important message is:

- **LB:** $q(L_k)$ is supervised / dual-source
- **YFB:** $q(L_k)$ is unsupervised / genomics-only

7.3 $q(F_k)$ in YFB

This is where YFB is mathematically richer than LB.

In YFB, F_k affects:

1. reconstruction $Y \approx LF^\top$
2. survival through $Z_F = YF$

So the intended update is dual-source:

$$A_{F,k}(j) = (1-\alpha)\tau_j \sum_i E[l_{ik}^2] + \alpha E[\tilde{\beta}_k^2] \sum_i w_i y_{ij}^2$$

$$B_{F,k}(j) = (1-\alpha)\tau_j \sum_i R_{ij}^{-k} E[l_{ik}] + \alpha E[\tilde{\beta}_k] \sum_i w_i z_i^{-k} y_{ij}$$

Then:

$$x_{F,jk} = \frac{B_{F,k}(j)}{A_{F,k}(j)}, \qquad s_{F,jk} = A_{F,k}(j)^{-1/2}$$

and an EBNM problem is solved across the entries of column F_k.

So mathematically:

- **LB:** $q(F_k)$ is genomics-only
- **YFB:** $q(F_k)$ is dual-source

This is the biggest derivational change.

8 What the Current Implementation Does Pragmatically

The intended YFB derivation says $q(F_k)$ is dual-source. But the current implementation does **not** actually use the survival contribution when updating F.

It sets:

$$\alpha_F = 0$$

so the implemented update reduces to:

$$A_{F,k}(j) = \tau_j \sum_i E[l_{ik}^2]$$

$$B_{F,k}(j) = \tau_j \sum_i R_{ij}^{-k} E[l_{ik}]$$

which is effectively pure genomics.

That is deliberate. The code keeps the survival term out of the F update for stability, while still learning $\tilde{\beta}$ from the observed YF scores.

So the honest description is:

- **mathematical YFB:** F should be supervised
- **implemented YFB right now:** F is kept unsupervised for stability, while $\tilde{\beta}$ is learned from the observed YF scores

That means current YFB is closer to:

- unsupervised factor learning for F
- supervised Cox fit on projected scores YF

than to a fully joint dual-source F update.

9 Why the YFB Loop Updates $\beta \to L \to F$

The current YFB loop does:

1. update β_k
2. update L_k
3. update F_k

inside each factor.

That differs from LB's more natural $L \to F \to \beta$ pattern.

The practical reason is:

- in YFB, β_k can already use the observed score $ZF[, k]$
- so it has signal immediately from the current F
- that makes it useful to update β first

This is more of an implementation choice than a core derivational claim.

10 ELBO Interpretation Under YFB

The survival ELBO is still computed by the same second-order Cox approximation.

For LB, that variance term is built from posterior uncertainty in L and β.

For YFB, the code passes:

- $EL \leftarrow ZF$
- $EL2 \leftarrow ZF^2$

into the survival ELBO call.

That means the survival uncertainty correction is only coming from uncertainty in $\tilde{\beta}$, not uncertainty in ZF itself, because ZF is treated as an observed score matrix at that iteration.

So:

- **LB survival uncertainty:** from uncertain latent L

- **YFB survival uncertainty:** from uncertain $\tilde{\beta}$, with ZF treated as observed conditional on the current F

11 Most Important Intuitive Takeaway

If you need a short advisor-facing explanation, use this:

- **LB** says: "learn latent patient scores L, then survival depends on those latent scores."
- **YFB** says: "survival should depend on the same factor-score projection YF that can be computed for both train and test patients."

That is why:

- L becomes genomics-only in YFB
- $\tilde{\beta}$ is learned on YF
- F becomes the critical bridge between reconstruction and prognosis

12 Clean Summary of What Changes in Each Variational Factor

12.1 $q(L)$

- **LB:** genomics + survival
- **YFB:** genomics only

12.2 $q(F)$

- **LB:** genomics only
- **YFB derivation:** genomics + survival
- **YFB current implementation:** genomics only ($\alpha_F = 0$)

12.3 $q(\beta)$

- **LB:** regress Cox working response on latent L_k
- **YFB:** regress Cox working response on observed projection score $ZF_k = (YF)_k$

That is the cleanest side-by-side comparison.

13 One Caveat to Keep in Mind

The current positive YFB results do **not** yet prove the fully derived YFB model is optimal.

What the current results support is narrower:

- the YFB survival parameterization $\eta = YF\tilde{\beta}$ is useful
- the train/test consistency idea is useful
- the current stable implementation works best when F is effectively learned from genomics and survival acts through $\tilde{\beta}$ on YF

So if you are asked whether the current code implements the full theoretical YFB derivation, the accurate answer is:

Not completely.

It implements the YFB survival parameterization and the corresponding $q(\beta)$ and $q(L)$ logic, but it currently suppresses the survival term in $q(F)$ for stability.